DATE: October 16, 2026

TO: Prof. Andreas Gerstlauer, Faculty Mentor
Nathanael Berhane, Writing TA

FROM: Team IH02
Henry Tudor, Dhwanit Kumawat, Ibraheem Moin, Tamim Hasan,
Ayhan Batoaq, Jacob Erattuparambil, Venthan Dinesh

SUBJECT: Design Proposal for AI Assisted Hardware Design from Python to FPGA

INTRODUCTION

This memo proposes an FPGA development and evaluation project for Ericsson, our sponsor. We will deliver a reproducible workflow for translating neural-network computations from Python into verified hardware on the AMD Kria KV260, and compare manual development with general AI coding assistance and ChipAgents. The intended users are Ericsson hardware and verification engineers evaluating where AI assistance can reduce implementation and debugging effort. Our proposed value to Ericsson is evidence for choosing a development approach, supported by working hardware and traceable engineering records [1].

The project addresses the effort required to turn a software model into correct, implementable RTL. Engineers must specify arithmetic, map computation to limited hardware, build tests, and integrate host communication. We will test whether AI assistance reduces that effort while meeting the same correctness and implementation requirements. Tiny ResNet-8 is the fall workload, with incremental acceleration targeted for December 7, 2026; accelerated ResNet-50 inference remains the spring goal for the April 2027 showcase [1].

The following sections define the workflow gaps, model and system scope, measurable acceptance criteria, deliverables, and individual responsibilities. Hardware latency and resource use will measure implementation quality; they will not replace development effort as the main criterion for comparing workflows. Appendix A explains the references and their relevance.

PROBLEM STATEMENT

The central question is whether an AI-assisted workflow can produce a verified field-programmable gate array (FPGA) component with less total engineering effort than

manual development. Producing plausible register-transfer-level (RTL) code is insufficient: the result must match an independent reference, meet interface requirements, fit the FPGA, and pass implementation checks. The hardware accelerator supplies a realistic test case for this comparison.

Our project will investigate three challenges in developing FPGA hardware. First, converting a software model into hardware requires decisions about bit widths, numerical scaling, and data storage. We will document these decisions so that teammates and AI tools follow the same rules. Second, code changes can introduce new bugs or leave existing problems unresolved. We will use shared tests and save test results to track whether each change works correctly. Third, AI may generate code quickly but still require substantial review and debugging. We will measure the total time and effort needed to produce a verified design, rather than code-generation time alone. Our evaluation will determine how much these challenges affect our project and whether the proposed workflow helps address them.

DESIGN PROBLEM DESCRIPTION

The design combines a residual-network workload, an FPGA execution system, and a controlled comparison of development approaches. The sections below explain the model, distinguish current evidence from unresolved engineering work, and define how each workflow will be evaluated.

Residual Network Workload

Our project uses image classification as a practical task for evaluating AI-assisted hardware development. Image classification assigns an image to a category based on its contents. A neural network performs this task by passing the image through layers of calculations. During training, the network learns numerical parameters called weights from labeled examples. During inference, it uses those learned weights to classify new images. Our project focuses on implementing and accelerating inference on an FPGA.

A convolutional neural network processes images using learned filters. Each filter examines small regions of an image and combines their pixel values to detect useful patterns. Early layers can identify simple features, such as edges, while later layers combine those features into more

complex patterns. Each layer produces intermediate results called feature maps. The number of channels describes how many different feature maps a layer contains.

A residual network, or ResNet, is a convolutional network that includes shortcut connections. A shortcut carries a block's input around several processing layers and adds it to their output. If the processing layers compute F(x), the block produces F(x) + x. When the input and output have different dimensions, a projection P(x) adjusts the shortcut so that the block can compute F(x) + P(x). These connections help make deep networks easier to train [2]. In our FPGA implementation, they also require hardware to preserve the shortcut data and coordinate the two paths so that matching values are available for addition.

We will begin with Tiny ResNet-8, a small residual network that provides a manageable starting point for hardware development. Our working model accepts 32 × 32 images with three color channels—red, green, and blue—and produces ten scores, one for each possible image category. Its structure includes an initial convolution, three residual blocks, global average pooling, and a classifier. Global average pooling reduces each feature map to its average value, and the classifier uses those values to produce the final category scores. The network's stages use 26, 52, and 104 feature channels [1]. The MLCommons image-classification implementation provides related model context [3].

Before implementing the model in hardware, we must define how its numerical values are represented. Our software references use both 32-bit floating-point arithmetic, called FP32, and quantized arithmetic based on 8-bit integers, called INT8. Quantization represents values with fewer bits, which can reduce storage requirements and simplify hardware arithmetic, but it can also reduce classification accuracy. The team's software results show 89.2% accuracy for FP32 and 87.9% for INT8 on the same 1,000-image subset, a decrease of 1.3 percentage points [1].

These software results provide a baseline for checking the FPGA implementation. We will keep the model configuration, trained weights, evaluation images, labels, and preprocessing consistent across tests. For each supported operation, we will compare the FPGA's integer outputs with an independent INT8 software reference. This comparison will help identify hardware errors separately from the accuracy loss already caused by quantization.

In the spring, we aim to extend the architecture toward ResNet-50, a deeper residual network with greater computation and memory requirements [2]. The architecture will reuse hardware compute units across supported layers. Host software running on the processor will prepare data, manage transfers, and execute any operations outside the FPGA's implemented scope. The final demonstration will identify which operations run on the FPGA and report performance for the complete inference process, including the required host work and data transfers.

System Architecture

The system will divide responsibilities between host software and FPGA logic. Host software will prepare images, load weights and configuration, launch accelerator work, retrieve tensors, and save results. FPGA logic will perform the selected arithmetic using a reusable convolution engine, buffering, and control. Figure 1 shows this boundary. The selected AXI control and data interfaces, register map, tensor layout, and transfer mechanism must be documented before integration [4], [5].

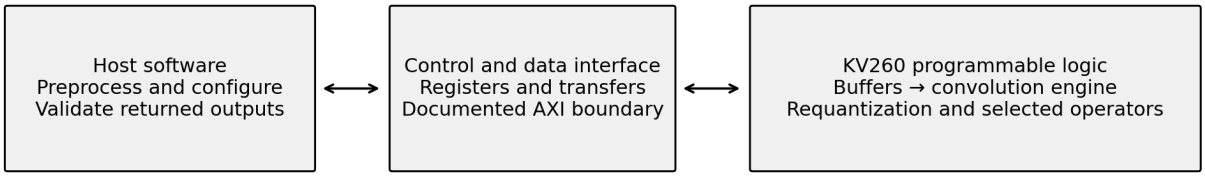


Figure 1. Proposed Host and FPGA System

The numerical contract will define tensor ordering, signedness, accumulator widths, scales, rounding, and saturation at every boundary. Batch normalization may be folded into inference parameters only when the software reference applies the same transformation. Residual addition requires matching dimensions and compatible scales. These rules will be shared across all development approaches so that a faster workflow cannot gain an advantage by implementing different behavior.

Work Investigated

The team has explored a fully parallel convolution architecture that exceeded available resources, a single multiply-accumulate (MAC) design, and the current 416-MAC engine [1]. This work establishes the practical tradeoff between parallelism and FPGA capacity. The 416-MAC engine is a candidate architecture, with timing closure and board validation remaining to be completed.

Working FP32 and INT8 references and pretrained weights provide the starting point for verification [1]. The software references and architecture exploration provide a foundation for further hardware development. They do not yet demonstrate that the full network runs in hardware or that one development workflow is superior.

Remaining Challenges

The first challenge is numerical correctness across convolution, requantization, activation, and residual addition. We will use independently maintained golden tensors, directed arithmetic cases, and repeatable random tests to locate discrepancies at operator boundaries. This approach prevents classification accuracy alone from hiding arithmetic errors.

The second challenge is fitting computation and intermediate data into the KV260 while meeting timing. Resource reports, timing analysis, and buffer-occupancy estimates will guide the degree of parallelism and compute reuse. Research on domain-specific inference hardware motivates measuring computation and memory behavior together, but its reported ASIC performance is not a prediction for our FPGA [6].

The third challenge is obtaining useful acceleration after communication and software overhead. We will measure kernel time separately from host-to-FPGA transfer, FPGA-to-host transfer, and complete application latency. A final challenge is comparison fairness: developer experience, prior solutions, tool access, and learning effects can change task duration. The evaluation controls below will make those limitations visible.

DEVELOPMENT WORKFLOW EVALUATION

Our evaluation focuses on implementing a predefined hardware architecture. The team will establish the microarchitecture, numerical behavior, interfaces, and design constraints before

each comparison task begins. Manual development, general AI coding assistance, and ChipAgents will each implement the same specification. We will compare the time and effort required to produce correct, implementable RTL, including review, testing, and debugging. Latency and FPGA resource use will provide additional measures of implementation quality. Figure 2 shows the common implementation and verification process used by all three approaches.

Development Approaches

In manual development, engineers read the specification, implement the specified microarchitecture, write RTL and supporting code, run simulation and synthesis, and diagnose failures themselves. Documentation, conventional editors, simulators, and EDA tools are allowed. Manual development provides the baseline for engineering effort and design quality.

Limited use of a separate AI chat tool is permitted for manual development in the use case of general explanations, documentation questions, and minor supporting tasks. AI tools integrated into the editor or terminal, including VS Code extensions and coding agents, are excluded from the manual workflow. Any permitted AI use will be recorded, and the time spent using it will count toward development effort.

Claude Code represents a general AI coding approach. Anthropic describes it as a tool that can work with a codebase, edit files, and run commands [7]. In our evaluation, an engineer will provide requirements and review proposed implementation or debugging changes. The record will identify the exact tool, model, permissions, and version. A substitute general coding tool must be reported as its own configuration rather than pooled silently with Claude Code.

ChipAgents represents a hardware-focused AI approach marketed for semiconductor design and verification [8]. We will evaluate the functions available in the team's actual installation, such as RTL development or debugging assistance, and record tool access and configuration. Vendor claims will provide product context, while our own tests will establish whether its output satisfies project requirements.

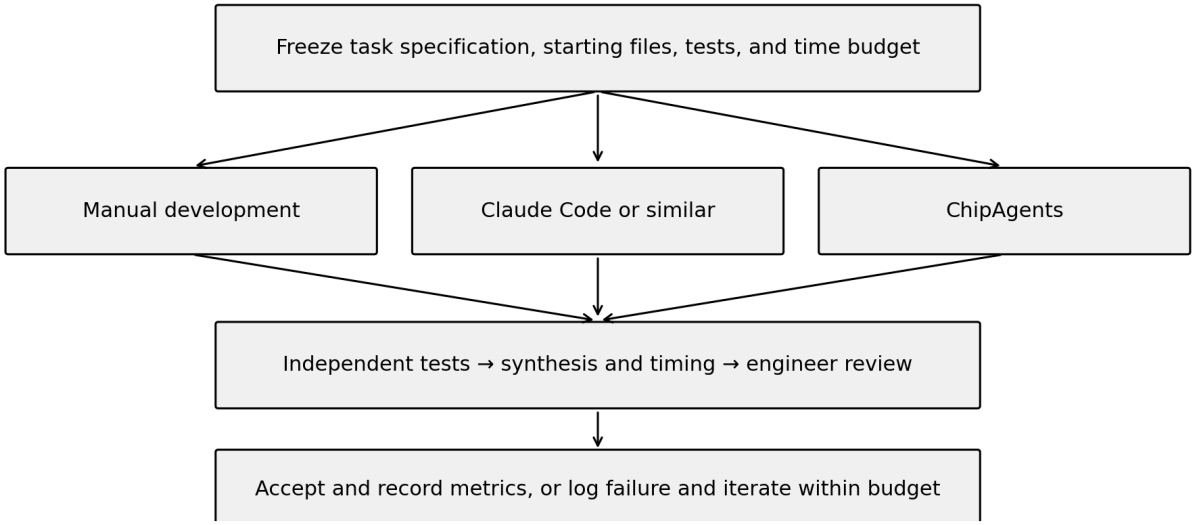


Figure 2. Common Workflow for the Three Development Approaches

Comparison Method and Success Criteria

The proposed evaluation contains at least three matched tasks, covering arithmetic RTL, interface or control logic, and debugging. Each approach receives the same frozen specification, initial files, public tests, tool constraints, and task-specific time budget. Acceptance tests will be maintained separately and withheld from implementation where practical. At least two independent attempts per approach and task will be sought, giving a target of 18 trials. Any reduced trial count or missing tool access will be reported.

A trial must meet its task-specific acceptance criteria before it is classified as successfully completed. For each task category, we will compare completion rates and median active human time across the three development approaches. The proposed productivity target is at least 20% lower median active human time than manual development, with no reduction in completion rate. Hardware quality will be evaluated using the timing, resource, and performance requirements established for that task. Results will identify any tradeoffs between development effort and implementation quality.

Active human time includes specification clarification during the task, prompting, coding, review, debugging, and test analysis. Wall-clock time includes AI response and tool waits, which will also be itemized. Completed trials will report both time measures and human interventions.

Unfinished trials will retain elapsed time and failure reasons; they will not be discarded or treated as quick successes. If results trade lower effort against higher hardware cost, the report will present that tradeoff instead of naming an unconditional winner.

AI records will include prompts, input and output token counts, cache-related usage when exposed, tool and model versions, and attributable usage cost when available. Unavailable usage will be marked unavailable, not zero. Tokens measure the cost of the development tool, not the image-classification workload. Hardware metrics and AI usage will be reported separately.

Task Acceptance Criteria

Table 1 defines the acceptance criteria established before the trials begin. All three development approaches will receive the same requirements for a given task. Development time, review and debugging effort, and available AI usage will be recorded for every trial.

Table 1. Task Acceptance Criteria

Task Category	Acceptance Criteria	Additional Measurements
Arithmetic RTL Implementation	Implement the specified arithmetic architecture, including bit widths, signedness, rounding, saturation, and pipeline behavior. Outputs must match the independent integer reference exactly for all designated directed and seeded random tests. The module must synthesize within the assigned resource budget and meet the specified clock constraint. For supported arithmetic operators, the designated suite will meet the minimum test counts specified in F2.	Cycle latency, initiation interval, implemented timing, and LUT, FF, DSP, and BRAM use.

Task Category	Acceptance Criteria	Additional Measurements
Interface or control logic implementation	Implement the specified register map, control sequence, and applicable interface protocol. Tests must confirm correct reset, configuration, start and completion behavior, back-to-back requests, and operation under stalls. The design must complete valid transactions without data loss, duplication, or deadlock and handle invalid requests according to the specification. The module must synthesize and meet its assigned timing and resource constraints.	Transaction completion time, sustained transfer rate where applicable, and timing and resource use.
Debugging	Correct a predefined defect in the supplied RTL while preserving the specified architecture and restoring the intended external behavior. The corrected design must pass the test that exposes the defect and the complete designated regression suite. The submission must explain the root cause, identify the correction, and include a regression test that fails on the original design and passes after the fix. The corrected RTL must synthesize and meet the task's assigned timing and resource constraints.	Time to identify the root cause, time to produce an accepted fix, number of attempted fixes, and review effort.

Passing these criteria determines whether a trial is complete. Hardware performance and resource measurements will be compared where relevant to the task. A failed or unfinished trial will retain its elapsed time, test results, and reason for failure.

REQUIREMENTS SPECIFICATION

The requirements below describe what the project aims to deliver and how acceptance will be checked. New numerical thresholds are proposed targets for team and sponsor review. They establish a measurable basis for comparison without representing preliminary estimates as demonstrated performance. Any later change must identify the reason and preserve results measured under the earlier specification.

The functional and hardware requirements below apply to the integrated accelerator system. Individual comparison trials will be evaluated against the requirements relevant to their assigned component and task category. Full-network accuracy, board communication, and end-to-end performance will be verified during system integration rather than required independently from every trial.

Functional and Interface Requirements

Table 2 defines the behavior required for an accepted accelerator component and its host interface. The same contracts apply to manually written and AI-assisted implementations.

Table 2. Functional and Interface Acceptance Criteria

ID	Required outcome	Acceptance evidence
F1	Accept the frozen Tiny ResNet-8 tensor shapes, weights, and quantization metadata; initial image shape is 32 × 32 × 3.	Versioned tensor schema and tests of valid and invalid shapes.
F2	Produce bit-exact integer outputs against the independent reference for every supported operator.	Zero mismatches across at least 100 directed and 1,000 seeded random cases per operator configuration, plus frozen model fixtures.
F3	Expose documented configuration, start, completion, status, and output retrieval behavior.	Register and transfer specification; tests for reset, back-to-back work, stalls, and invalid configuration.

ID	Required outcome	Acceptance evidence
F4	Transfer tensors without data corruption or deadlock.	At least 100 consecutive board transactions with zero mismatches or timeouts; timeout threshold recorded per test.
F5	Provide a command-line runner that loads data, invokes the selected path, saves outputs and timing, and returns errors explicitly.	A teammate reproduces the documented example; invalid input and timeout tests return nonzero status.

Hardware and Measurement Requirements

Table 3 defines implementation quality and measurement boundaries. A proposed 100 MHz accelerator clock provides a common starting constraint. End-to-end speedup above 1.0 is an acceleration objective for a declared selected workload. Missing it is a reportable hardware outcome and does not invalidate an otherwise controlled workflow comparison.

Table 3. Hardware and Performance Acceptance Criteria

ID	Required outcome	Acceptance evidence
P1	Fit the selected KV260 device and close timing at the common proposed 100 MHz clock.	Implemented LUT, FF, DSP, and BRAM usage within device capacity; worst setup slack ≥ 0 ns and no unresolved timing or implementation violations.
P2	Preserve reference predictions when selected operators are offloaded.	Zero changed predictions relative to the frozen INT8 reference on the same 1,000-image subset; report FP32, INT8, and accelerated accuracy separately.
P3	Measure latency, throughput, transfers, and CPU comparison using fixed workloads and settings.	At least 10 warm-up and 100 timed invocations; median and 95th-percentile latency, images/s, clock, batch size, CPU model, and thread count.

ID	Required outcome	Acceptance evidence
P4	Demonstrate useful acceleration on a declared workload, with a proposed speedup target > 1.0.	CPU time divided by accelerated time for identical computation; report kernel and end-to-end ratios, including transfer and host work.
P5	Make builds and experiments reproducible from a tagged release.	A teammate rebuilds, runs the regression suite, and regenerates result summaries using recorded tools, seeds, weights, and commands.

For full inference, end-to-end timing begins with a decoded input image before model-specific preprocessing and ends when class outputs are available to the host. All model-specific preprocessing, transfers, launches, and required host operators within that boundary are included. Initial model loading and build time will be reported separately. For operator tests, both CPU and FPGA measurements will use the same operator inputs and outputs. Throughput will be measured over the completed run, rather than inferred from peak MAC count.

Workflow and Delivery Requirements

Table 4 connects the comparison to concrete records. The final recommendation must remain useful even if AI assistance fails to meet the proposed productivity target.

Table 4. Workflow Acceptance Criteria

ID	Required outcome	Acceptance evidence
W1	Compare all three development approaches on common tasks.	Target of three matched tasks and two attempts per approach per task; disclose missing trials and access limitations.
W2	Account for complete development effort and AI usage.	Per-trial active human and wall-clock time, review/debug time, interventions, test failures, tool waits, and available token/cost records.

ID	Required outcome	Acceptance evidence
W3	Apply the same task-specific acceptance criteria to all three approaches before comparing productivity.	Pass/fail results by task category, completion rates, assessment of the proposed 20% reduction in active human time, and relevant hardware-quality measurements.
W4	Deliver source, runnable tests, board artifacts, and evaluation records.	Tagged release containing all deliverables in Table 5, with a documented reproduction check.

DELIVERABLES

The deliverables are specific engineering artifacts that another engineer can inspect and run. Table 5 separates the fall scope from the spring extension. The minimum fall hardware commitment is one verified convolution on the board, with additional operators identified explicitly as scope expands.

Table 5. Project Deliverables

Artifact	Fall delivery by December 7 2026	Spring extension by April 2027
RTL and board implementation	Convolution engine, necessary control/buffers, constraints, build scripts, bitstream, and implementation reports.	Reusable operator configurations and scheduling for accelerated ResNet-50 inference; declared FPGA/host partition.
Software and model package	FP32 and independent INT8 references, preprocessing, parameter export, checkpoint provenance, tensor schema, and host runner.	ResNet-50 reference and parameters, compatible host execution, and end-to-end inference runner.
Verification suite	Directed and seeded tests, golden tensors, operator regression, interface tests, and board comparison scripts.	Additional operator/layer fixtures and complete inference checks for the spring scope.

Artifact	Fall delivery by December 7 2026	Spring extension by April 2027
Workflow comparison package	Task specifications, approach configurations, trial logs, available token usage, and initial comparison results.	Completed evaluation, raw measurements, limitations, and recommendations for Ericsson engineers.
Documentation and demo	Architecture/interface specification, setup instructions, measured fall demo, and reproducible release.	Showcase demo, final report or paper, and release with build and evaluation instructions.

STANDARDS AND ENGINEERING RESPONSIBILITIES

The implementation will follow the language, protocol, and platform rules relevant to its actual configuration. SystemVerilog will use the synthesis tool's supported subset of IEEE 1800-2023, with explicit widths and signedness in arithmetic and documented reset behavior [9]. AXI interfaces will follow the selected Arm protocol revision; the specification will identify whether each connection uses AXI4, AXI4-Lite, or AXI4-Stream and cite its applicable document [4].

The KV260 setup will follow AMD guidance for power, cooling, clocks, and supported board operation [5]. Source and tool versions, pin and timing constraints, and interface revisions will be recorded in the release. The project's numerical contract is an internal engineering specification, not an external standard. Use of MLCommons material does not establish official MLPerf compliance [3].

ETHICS AND SAFETY

We will preserve licenses and attribution for software, RTL, weights, datasets, and third-party modules. AI-generated contributions will be identified in development records and reviewed by a teammate before integration. Sponsor-confidential material will be submitted to an external AI service only when authorized. These responsibilities apply to every workflow, including prompt and diagnostic logs retained for evaluation.

Reports will distinguish measurements, estimates, proposed targets, and unsuccessful trials. The board will be operated within manufacturer limits, with appropriate cooling and power procedures [5]. The demonstration is a research image-classification system; it will not be

presented as validated for safety-critical use. Engineers retain responsibility for verification and final design decisions.

TEAM ORGANIZATION

Team IH02 has seven members, with Prof. Andreas Gerstlauer as faculty mentor and Ericsson as sponsor [1]. Table 6 proposes individual ownership within the hardware and software sub-teams. Each owner is responsible for the named artifact and its tests; integration and review remain shared responsibilities.

Individual Responsibilities

Assignments follow the boundaries between reference behavior, arithmetic hardware, system integration, and workflow measurement. This division gives each interface an identifiable owner and makes handoffs between sub-teams explicit.

Table 6. Proposed Individual Responsibilities

Member	Primary project ownership	Required handoff	Related coursework
Henry Tudor *Software*	FP32 reference, image preprocessing, dataset split, and classification accuracy evaluation.	Frozen inputs/labels and accuracy script for Dhwanit and RTL owners.	Software Design and Implementation II, VLSI, Computer Architecture
Dhwanit Kumawat *Software*	INT8 numerical contract, quantization, weight/scale export, and independent integer golden tensors.	Versioned parameter package and operator fixtures for Ayhan and Jacob.	Software Design and Implementation II, VLSI, Computer Architecture, Performance Evaluation/Benchmarking
Ibraheem Moin *Hardware and project lead*	Top-level operator integration, synthesis flow, resource tracking, and technical coordination.	Integrated build and reports for Venthan; scope/dependency decisions for the team.	HDL, VLSI, HW/SW Co-Design, Computer Architecture, Performance Benchmarking

Member	Primary project ownership	Required handoff	Related coursework
Tamim Hasan *Software and evaluation*	Matched-task definitions, time/intervention logs, AI usage records, and comparison analysis.	Reproducible evaluation tables and evidence for the final report; task records from every member.	Software Design and Implementation II, HDL, Computer Architecture
Ayhan Batoaq *Hardware*	Compute microarchitecture, MAC scheduling, buffering, and data reuse.	Documented compute/buffer interfaces and unit tests for Jacob and Ibraheem.	HDL, VLSI, ASIC, Performance Evaluation/ Benchmarking, Computer Architecture, Multithreaded Architectures
Jacob Erattuparambil *Hardware*	Convolution and residual arithmetic RTL, requantization/ReLU integration, simulation, and timing review.	Verified operator RTL, regression evidence, and timing findings for Ibraheem.	HDL, VLSI, Microarchitecture
Venthan Dinesh *Hardware and System Integration*	KV260 control/data interface, host launch/transfer integration, board deployment, and release scripts.	Runnable board example, transfer tests, and measured transfer overhead.	HDL, VLSI, ASIC, SoC

Coordination and Review

Monday meetings will review progress, blockers, and upcoming dependencies. Each issue will name one owner, a deliverable, acceptance evidence, and a due date. Ibraheem will consolidate technical decisions and questions for the mentor and sponsor. Tamim will maintain the evaluation ledger, while each developer remains responsible for submitting complete records.

The shared GitHub repository will have version specifications, source, tests, parameters, and results. A teammate must review changes before integration. RTL reviews require relevant

regression evidence; interface or numerical changes require review by both the producer and consumer. Jacob will maintain release instructions, and another teammate will perform the reproduction check.

Technical disagreements will be resolved using correctness, implementation, performance, and effort evidence. Where needed, we will run a bounded comparison before deciding. Missed commitments require an updated issue and explicit handoff. Unresolved technical or participation problems will be raised with the faculty mentor after team discussion. Scope changes will identify their effect on requirements, verification, and milestones.

CONCLUSION

The project will provide Ericsson with a working FPGA demonstration and evidence comparing manual development, general AI coding assistance, and ChipAgents for implementing a predefined hardware architecture. Success requires correct hardware, reproducible deliverables, and a fair comparison that accounts for implementation, review, testing, and debugging effort. Each trial will be assessed against the acceptance criteria for arithmetic RTL, interface or control logic, or debugging. The proposed AI productivity target is at least 20% lower median active human time than manual development within each task category, with no reduction in completion rate. The evaluation will support a recommendation even if this target is not met.

The main challenges are maintaining numerical correctness, meeting FPGA resource and timing constraints, managing communication overhead, and comparing workflows fairly. Independent integer references and operator tests will help identify arithmetic errors. Implementation reports will show whether the specified architecture meets its constraints, while separate kernel and system measurements will reveal transfer overhead. Common task specifications, consistent acceptance tests, and complete development records will support comparison across the three approaches.

The next steps are to finalize the Tiny ResNet-8 model checkpoint, numerical rules, and architecture specification, then confirm the task-specific acceptance criteria and proposed targets with the mentor and sponsor. Jacob and Ayhan will verify the convolution engine, Ibraheem will review synthesis and timing, and Venthan will establish the board interface and transfer baseline.

Henry and Dhwanit will complete the software references and test data, while Tamim will prepare the comparison tasks and measurement records. Following successful convolution verification, the team will integrate requantization and ReLU, demonstrate board execution, and expand the verified implementation toward the fall deliverables and spring ResNet-50 objective.

REFERENCES

[1] Team IH02, "AI-assisted hardware design: Python to FPGA," project proposal, Oct. 2, 2026, 95bc6015274dbfbea60698ae7592e4bb3d1631f0. Available: https://github.com/jacob-eratt/senior-design/tree/95bc6015274dbfbea60698ae7592e4bb3d1631f0

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," arXiv:1512.03385, 2015. Available: https://arxiv.org/abs/1512.03385

[3] MLCommons, "Tiny image-classification model," keras_model.py, MLPerf Tiny repository. Available: https://github.com/mlcommons/tiny/blob/master/benchmark/training/image_classification/keras_model.py

[4] Arm, "AMBA AXI protocol specification," IHI 0022. Applicable interface revision to be recorded in the project specification. Available: https://developer.arm.com/documentation/ihi0022/latest/

[5] AMD, "Kria KV260 Vision AI Starter Kit User Guide," UG1089. Available: https://docs.amd.com/r/en-US/ug1089-kv260-starter-kit

[6] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," arXiv:1704.04760, 2017. Available: https://arxiv.org/abs/1704.04760

[7] Anthropic, "Claude Code," product documentation. Available: https://claude.com/product/claude-code

[8] ChipAgents, "The agentic foundation for semiconductor design," product website. Available: https://chipagents.ai/

[9] IEEE, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2023. Available: https://standards.ieee.org/ieee/1800/7743/

APPENDIX A REFERENCE CONTEXT AND APPLICATION

The notes below explain how each reference supports this proposal and where its evidence ends. Project measurements come from the team's software evaluations and design investigations. External sources provide model, platform, protocol, or tool context.

Project records [1]

The project records document the team organization, model configuration, software accuracy results, and convolution architectures investigated. The final release will identify the exact model checkpoint, evaluation dataset, code revision, and measurement records.

Residual learning [2]

He and colleagues describe residual learning and the use of shortcut connections in deep image-recognition networks. The paper supports the explanation of residual blocks and the larger ResNet family. It does not establish the architecture, accuracy, or FPGA performance of the team's Tiny ResNet-8 implementation.

MLCommons model context [3]

The MLPerf Tiny repository provides an image-classification model implementation useful for understanding a small residual-network workload and tracing any reused model code. The team must freeze the actual source revision and parameter checkpoint used locally. A shared model lineage does not imply that the team's configuration is identical or that its results satisfy MLPerf submission rules.

AXI interface rules [4]

Arm's AXI specification defines protocol behavior relevant to memory-mapped communication. The interface owner must record the exact applicable revision and selected protocol. If the design uses AXI4-Stream, its separate applicable specification must also be identified. Register behavior, tensor packing, and timeout policy remain project-specific requirements that need their own documentation and tests.

KV260 platform guidance [5]

AMD UG1089 provides the platform context for setting up and operating the KV260. It informs board preparation, tool integration, and operating constraints. Actual device selection, resource

capacity, pin assignment, and implemented timing must be established from the selected board configuration and implementation reports; a board guide alone is not evidence that the candidate engine fits.

Inference architecture context [6]

Jouppi and colleagues evaluate a domain-specific inference ASIC and discuss compute and memory organization. The paper motivates treating arithmetic throughput and memory behavior together when evaluating an accelerator. Its hardware, workloads, and reported speedups differ from this project, so those numerical results are not transferred into our FPGA targets.

General AI coding assistance [7]

Anthropic's product documentation describes Claude Code as a coding tool that can operate on repository files and execute commands. This supports its role as the general coding-assistance comparison. Product descriptions do not establish RTL correctness or productivity on the team's tasks; those outcomes require independent tests and complete effort records.

Hardware-focused AI assistance [8]

ChipAgents describes an AI platform for semiconductor design and verification. This supports the choice of a domain-focused comparison approach. The team will document the capabilities it can actually access, and will evaluate accepted outputs rather than adopt vendor productivity claims. Tool availability and exposed usage accounting may limit the comparison and must be disclosed.

SystemVerilog language rules [9]

IEEE 1800-2023 defines SystemVerilog language semantics used in design and verification. Explicit widths, signed arithmetic, scheduling, and reset conventions are especially relevant to quantized datapaths. The implementation must still stay within the selected simulator and synthesis tool's supported subset; language compliance alone does not demonstrate synthesis or numerical correctness.